API: ESERCITAZIONI

COMPUTABILITÀ

Gabriele Paveri-Fontana

Mail: gabriele.paveri@polimi.it

Recap Computabilità

- Computabilità = risolvibilità meccanica
  - Calcolabilità di funzioni
  - Decidibilità di problemi

Recap Enumerazioni

- Una *enumerazione* di un insieme X è una corrispondenza biunivoca (biezione) tra X e l'insieme $\mathbb{N}$ dei numeri naturali:

$$X \overset{(1:1)}{\leftrightarrow} \mathbb{N}$$

N.B.: Indicheremo una biezione semplicemente con

$$\leftrightarrow$$

- Una *enumerazione effettiva* (*algoritmica*) è un'enumerazione che può essere computata da un algoritmo.

Recap Funzioni

- Consideriamo solo funzioni $f: \mathbb{N} \to \mathbb{N}$.
- E' una grossa limitazione? Assoltamente no!
  - $f: D \to C,\ D \leftrightarrow \mathbb{N},\ C \leftrightarrow \mathbb{N} \quad \Rightarrow \quad f: \mathbb{N} \to \mathbb{N}$
  - $\mathbb{N}^k \leftrightarrow \mathbb{N} \quad \forall k \in \mathbb{N}$
  - $\mathbb{Q} \leftrightarrow \mathbb{N}$ e quindi anche $\mathbb{Q}^k \leftrightarrow \mathbb{N}$
  - Ed $\mathbb{R}$? Non ci interessa perché gli irrazionali non sono esprimibili computazionalmente!
- Le funzioni $f: \mathbb{N} \to \mathbb{N}$ racchiudono tutto ciò che possiamo (sperare di) calcolare.

Recap Funzioni

- In realtà consideriamo funzioni $f: \mathbb{N} \rightarrow \mathbb{N} \cup \{\perp\}$
  - $D_f = D(f) = dom(f) \coloneqq \{x \in \mathbb{N} | f(x) \neq \perp\}$
  - $I_f = I(f) = imm(f) \coloneqq \{y \in \mathbb{N} | \exists x \in \mathbb{N} \text{ t.c. } f(x) = y\}$
- f è detta *totale* se $D(f) = \mathbb{N}$
- f è detta *parziale* se $D(f) \subset \mathbb{N}$

Recap Funzioni calcolabili

- Un *algoritmo* è una sequenza astratta di comandi per risolvere un problema in modo meccanico.
- Una funzione $f: \mathbb{N} \to \mathbb{N} \cup \{\perp\}$ è *calcolabile* se esiste un algoritmo che la calcola.
- **Tesi di Church (I)** Non esiste meccanismo di calcolo automatico più potente delle TM.
- **Tesi di Church (II)** Ogni algoritmo può essere codificato mediante una TM.
- Una funzione $f: \mathbb{N} \to \mathbb{N} \cup \{\perp\}$ è calcolabile se esiste una TM M che la calcola, ovvero tale che $f = \tau_M$.

Recap Funzioni calcolabili

- Un formalismo che abbia lo stesso potere espressivo delle TM è detto *Turing-completo*.
- Esistono svariati formalismi Turing completi: ad esempio linguaggi di programmazione come Java, C , FORTRAN.
- La calcolabilità di una funzione può essere stabilita con uno qualunque dei formalismi Turing completi.
- N.B.: **calcolabile** ≠ **calcolato**, ovvero il fatto di saper dimostrare che esiste una TM che calcola una certa funzione non implica che si riesca ad esibire tale TM, e dunque che si sappia calcolare la funzione.

Recap Enumerazione delle TM

- Le TM sono *algoritmicamente enumerabili*

$\mathcal{E}$: $TM \leftrightarrow \mathbb{N}$

enumerazione effettiva

- $\mathcal{E}$ è detta *goedelizzazione*
- $\mathcal{E}(M)$ è detto *numero di Goedel* di M
- M_y è la TM il cui numero di Goedel è y
- f_y è la funzione calcolata da M_y

Quindi $\mathbb{N} \leftrightarrow TM \leftrightarrow \{\text{funzioni calcolabili}\}$

- Adottiamo la convenzione che $f_y(x) = \perp$ se e solo se M_y non si ferma quando riceve in ingresso x (ovvero mandiamo le TM in loop infinito ogni volta che si fermerebbero in stato non accettante).

Recap TM riconoscitori VS TM calcolatori

- TM come *calcolatori di funzioni*:

  Sia $\{M_i | i \in \mathbb{N}\}$ una enum. effettiva delle TM che calcolano funzioni.
  Allora M_i calcola f_i: $\mathbb{N} \to \mathbb{N} \cup \{\perp\}$, ovvero:

  Input x
  - se $x \in D_{f_i}$, allora $M_i(x) = f_i(x)$ — Cioè M_i termina la computazione e restituisce $f_i(x)$
  - se $x \notin D_{f_i}$, allora $M_i(x) = \perp$ — Cioè M_i non termina la computazione (va in «loop»)

- TM come *riconoscitori di linguaggi*:

  Sia $\{M_i | i \in \mathbb{N}\}$ una enum. effettiva delle TM che riconoscono linguaggi.
  Allora M_i riconosce $L(M_i) \subseteq \mathbb{N}$, ovvero:

  Input x
  - se $x \in L(M_i)$, allora $M_i(x) =$ "Sì" — Cioè M_i termina la computazione e restituisce «Sì»
  - se $x \notin L(M_i)$, allora $M_i(x) = \perp$ — Cioè M_i non termina la computazione (va in «loop»)

Recap TM riconoscitori VS TM calcolatori

- Le due versioni delle TM sono equivalenti.
  - Si può affrontare il problema di riconoscere un linguaggio $L \subseteq \mathbb{N}$ tramite le TM calcolatori di funzioni. Infatti:

$$L \text{ è riconoscibile} \iff \sigma_L(x) = \begin{cases} 1 \text{ se } x \in L \\ \perp \text{ se } x \notin L \end{cases} \text{ è calcolabile}$$

  - Si può affrontare il problema di calcolare una funzione $f\colon \mathbb{N} \to \mathbb{N} \cup \{\perp\}$ tramite le TM riconoscitori di linguaggi. Infatti:

$$f \text{ è calcolabile} \iff L_f = \{x\$y \mid y = f(x)\} \text{ è riconoscibile}$$

Recap TM riconoscitori VS TM calcolatori

L è riconoscibile $\iff$ $\sigma_L(x) = \begin{cases} 1 \text{ se } x \in L \\ \perp \text{ se } x \notin L \end{cases}$ è calcolabile

($\Rightarrow$)

- L riconoscibile $\implies$ $\exists i \in \mathbb{N}$ t.c. $L = L(M_i)$
- Si consideri il seguente algoritmo:
  - Input x
  - Lancia la computazione di $M_i(x)$
  - Se la computazione si ferma (restituendo "Sì"), restituisce 1
- L'algoritmo descritto computa la funzione seguente:

$$x \mapsto \begin{cases} 1 \text{ se } x \in L \\ \perp \text{ se } x \notin L \end{cases}$$

Recap TM riconoscitori VS TM calcolatori

L è riconoscibile $\iff$ $\sigma_L(x) = \begin{cases} 1 \text{ se } x \in L \\ \perp \text{ se } x \notin L \end{cases}$ è calcolabile

$(\Leftarrow)$

- σ_L calcolabile $\implies$ $\exists i \in \mathbb{N}$ t.c. $\sigma_L = f_i$, ovvero $\sigma_L(x) = M_i(x)$
- Si consideri il seguente algoritmo:
  - Input x
  - Lancia la computazione di $M_i(x)$
  - Se la computazione si ferma (restituendo 1), restituisce "Sì"
- L'algoritmo descritto esegue la seguente computazione:

$$x \mapsto \begin{cases} \text{"Sì" se } x \in L \\ \perp \quad \text{se } x \notin L \end{cases}$$

Recap TM riconoscitori VS TM calcolatori

f è calcolabile $\iff$ $L_f = \{x\$y \mid y = f(x)\}$ è riconoscibile

($\Rightarrow$)

- f calcolabile $\implies$ $\exists i \in \mathbb{N}$ t.c. $f = f_i$, ovvero $f(x) = M_i(x)$
- Si consideri il seguente algoritmo:
  - Input $x\$y$
  - Lancia la computazione di $M_i(x)$
  - Se la computazione si ferma, restituendo y', controlla se $y' = y$, e quindi restituisce "Sì", oppure se $y' \neq y$, e quindi va in loop
- L'algoritmo descritto esegue la seguente computazione:

$$x\$y \mapsto \begin{cases} \text{"Sì"} & \text{se } y = f(x) \\ \perp & \text{se } y \neq f(x) \end{cases}$$

N.B.: questa condizione è verificata sia se $f(x) = y' \neq y$, sia se $f(x) = \perp$

Recap TM riconoscitori VS TM calcolatori

f è calcolabile $\iff$ $L_f = \{x\$y \mid y = f(x)\}$ è riconoscibile

($\Leftarrow$)

- L_f riconoscibile $\implies$ $\exists i \in \mathbb{N}$ t.c. $L_f = L(M_i)$
- Si consideri il seguente algoritmo:
  - Input x
  - Lancia la computazione di $M_i(x\$y)$ $\forall y \in N$ tramite enumerazione diagonale
  - Se la computazione si ferma a partire da $x\$y$, restituisce y
- L'algoritmo descritto computa la funzione seguente:

$$x \mapsto \begin{cases} y & \text{se } y = f(x) \\ \perp & \text{se } \nexists y' \text{ t.c. } f(x) = y' \end{cases}$$

Recap Problemi di decisione

- Vediamo un *problema di decisione* come un sottoinsieme dei numeri naturali $S \subseteq \mathbb{N}$.
- Un'*istanza* del problema S è "$x \in S$" con x numero naturale: la risposta può essere vero o falso.
- Possiamo associare ad ogni problema S la sua *funzione caratteristica* $\chi_S: \mathbb{N} \to \{0,1\} \subseteq \mathbb{N}$ definita da:

$$\chi_S(x) = \begin{cases} 1 \text{ se } x \in S \\ 0 \text{ se } x \notin S \end{cases}$$

- Per ogni $S \subseteq \mathbb{N}$, χ_S è una funzione totale per definizione. Tuttavia, non è detto che χ_S sia calcolabile!

Recap Decidibilità: definizione formale

- Un problema S è *decidibile* (o *ricorsivo*) se la sua funzione caratteristica χ_S è computabile.
  - Se χ_S è computabile, allora si può determinare algoritmicamente sia se $x \in S$ sia se $x \notin S$
  - Se χ_S è computabile, allora lo è anche χ_{S^c}, dunque S è decidibile sse S^C lo è
- Un problema S è *semidecidibile* (o *ricorsivamente enumerabile*) se $S = \emptyset$ oppure $S = I_f$ con f totale e computabile, ovvero si può enumerare algoritmicamente tutti e soli gli elementi di S.
  - Se si riescono a enumerare algoritmicamente gli elementi di S, non è detto che si riesca a fare altrettanto con gli elementi di S^C
  - Un'enumerazione algoritmica di S permette di determinare algoritmicamente se $x \in S$, ma non se $x \notin S$
- Un problema S è *indecidibile* se non è decidibile. (!!!)

Recap Decidibilità: definizione informale

- Un problema S è *decidibile* se è possibile definire un algoritmo che decida l'appartenenza ad S, ovvero:
  - se $x \in S$ termina con SI'
  - se $x \notin S$ termina con NO
- Un problema S è *semidecidibile* se è possibile definire un algoritmo che semidecida l'appartenenza ad S, ovvero:
  - se $x \in S$ termina con SI'
  - se $x \notin S$ non termina

Recap Decidibilità: teoremi

$$\{\text{insiemi ricorsivi}\} \subseteq \{\text{insiemi ric. enum.}\} \subseteq \wp(\mathbb{N})$$

- **Teorema1** S ricorsivo $\Rightarrow$ S ricorsivamente enumerabile
- **Teorema2** S ricorsivo $\Leftrightarrow$ S e S^c ricorsivamente enumerabili
- **Corollario1** S non ricorsivo, S^c ric.enum. $\Rightarrow$ S non ric.enum.
- **Teorema3** S ric.enum. $\Leftrightarrow$ $S = D_g$ con g computabile e parziale
- **Teorema4** S ric.enum. $\Leftrightarrow$ $S = I_h$ con h computabile e parziale
- **Corollario2** S ric.enum. $\Leftrightarrow$ la funzione σ_s è calcolabile, dove

$$\sigma_S(x) = \begin{cases} 1 \text{ se } x \in S \\ \perp \text{ se } x \notin S \end{cases}$$ *funzione semicaratteristica* di S

Recap Dimostrazione Teorema3

$S = \emptyset \vee S = I_f$ con f totale e computabile $\Leftrightarrow$ $S = D_g$ con g parziale e computabile

$(\Rightarrow)$

- Se $S = \emptyset$ allora basta porre $g(x) = \perp \ \forall x \in \mathbb{N}$.
- Se $S = I_f$ con f totale e computabile, allora poniamo:

$$g(x) = \begin{cases} 0 \text{ se } \exists i \text{ t. c. } f(i) = x \\ \perp \text{ altrimenti} \end{cases}$$

Osserviamo che g è computabile (enumerazione diagonale) ed è parziale.

Recap Dimostrazione Teorema3

$S = \emptyset \vee S = I_f$ con f totale e computabile $\Leftrightarrow$ $S = D_g$ con g parziale e computabile

$(\Leftarrow)$

- Se $g(x) = \perp \forall x \in \mathbb{N}$, allora $S = D_g = \emptyset$
- Altrimenti, sia k tale che $g(k) \neq \perp$ e sia i t.c. $g = f_i$.

  Definiamo:

$$f(n) = \begin{cases} x & \text{se } n = d(x,y) \text{ e la computazione della} \\ & \text{TM } M_i \text{ su input } x \text{ termina entro } y \text{ mosse} \\ k & \text{altrimenti} \end{cases}$$

dove $d(x,y) = \frac{(x+y)(x+y+1)}{2} + x$ è biezione effettiva tra $\mathbb{N}^2$ ed $\mathbb{N}$.

Recap Dimostrazione Teorema4

$S = \emptyset \vee S = I_f$ con f totale e computabile $\Leftrightarrow$ $S = I_h$ con h parziale e computabile

$(\Rightarrow)$
- Se $S = \emptyset$ allora basta porre $h(x) = \perp \; \forall x \in \mathbb{N}$.
- Se $S = I_f$ basta osservare che una funzione totale e computabile è un caso particolare di funzione parziale e computabile.

$(\Leftarrow)$
- Se $h(x) = \perp \; \forall x \in \mathbb{N}$, allora $S = I_h = \emptyset$
- Altrimenti, sia k tale che $h(k) \neq \perp$, sia $l = h(k)$ e sia i t.c. $h = f_i$. Definiamo:

$$f(n) = \begin{cases} h(x) & \text{se } n = d(x,y) \text{ e la computazione della TM} \\ & M_i \text{ su input } x \text{ termina entro } y \text{ mosse} \\ l & \text{altrimenti} \end{cases}$$

Recap Problemi finiti

- Sia $S \subseteq \mathbb{N}$ un problema *finito* (ovvero $|S| < +\infty$). Allora la sua funzione caratteristica assume sempre il valore 0 tranne che per un numero finito di punti.
- Una tale funzione è sicuramente calcolabile. Dunque…
- Qualunque problema finito è sempre (banalmente) decidibile!
- Lo stesso vale per i problemi *cofiniti*, ovvero il cui complementare sia finito (ovvero $|S^C| < +\infty$).

Recap Problemi in forma chiusa

- Un problema è *in forma chiusa* se non prevede istanziazioni su $x \in \mathbb{N}$, ma corrisponde ad un'unica domanda con risposta SI'/NO.
  Ad esempio:
  - I pianeti del sistema solare sono 9?
  - Una certa TM M data è tale che $L(M) = \emptyset$?
  - Due specifici programmi dati calcolano la stessa funzione?
- Il sottoinsieme S di $\mathbb{N}$ associato a un problema in forma chiusa è sempre $\mathbb{N}$ stesso oppure $\emptyset$. Se è $S = \mathbb{N}$, la funzione caratteristica χ_S è la funzione costante di valore 1; se $S = \emptyset$, χ_S è la funzione costante di valore 0. In entrambi i casi, si tratta di una funzione ovviamente calcolabile (ma non calcolata). Dunque…
- Qualunque problema in forma chiusa è sempre (banalmente) decidibile!

Recap Teorema di Rice

- Sia F un insieme di funzioni calcolabili. Allora:

$$S \coloneqq \{y \in \mathbb{N} \mid f_y \in F\} \text{ è decidibile}$$

$$\Updownarrow$$

$$F = \emptyset \text{ oppure } F = \{\text{funzioni calcolabili}\}$$

- ATTENZIONE: per poter applicare il teorema, l'insieme S deve essere tale che, per ogni $y \in S$, per ogni $y' \in \mathbb{N}$ tale che $f_y = f_{y'}$, anche $y' \in S$, ovvero: se S contiene il numero di Goedel di una TM, allora deve contenere anche i numeri di Goedel di tutte le TM ad essa equivalenti (che sono infinite…)!

Recap Riduzione di problemi

- S è *riducibile* a S' (si indica $S \leq_r S'$) sse esiste una funzione totale e calcolabile t tale che per ogni $x \in \mathbb{N}$

$$x \in S \Leftrightarrow t(x) \in S'$$

N.B.: t è detta *riduzione*

- $S \leq_r S'$, S' ricorsivo $\Rightarrow$ S ricorsivo
- $S \leq_r S'$, S' ric.enum. $\Rightarrow$ S ric.enum.

- $S \leq_r S'$, S non ricorsivo $\Rightarrow$ S' non ricorsivo
- $S \leq_r S'$, S non ric.enum. $\Rightarrow$ S' non ric.enum.

Recap Riduzione di problemi

- Un caso particolare di riduzione è dato da *generalizzazioni* e *particolarizzazioni*:
  - Se un problema S è un caso particolare di un problema S', allora si ha $S \leq_r S'$
  - Se un problema S è una generalizzazione di un problema S', allora si ha $S' \leq_r S$

Recap Tecniche di risoluzione dei quesiti

- Riassumiamo le tecniche utilizzabili nello svolgere un esercizio di calcolabilità di una funzione o di (semi)decidibilità di un problema:
  - Applicazione diretta delle definizioni
  - Teoremi sulla decidibilità
  - Finitezza e forma chiusa
  - Teorema di Rice
  - Riduzioni
  - Diagonalizzazione

Recap Tecniche di risoluzione dei quesiti

- Per dimostrare che una funzione è calcolabile:
  - Esibire un algoritmo che la calcola
- Per dimostrare che una funzione non è calcolabile:
  - Tecniche di diagonalizzazione
  - Teorema di Rice (per funzioni caratteristiche)
  - Riduzioni (per funzioni (semi)caratteristiche)

Recap Tecniche di risoluzione dei quesiti

- Per dimostrare che un problema S è decidibile:
  - Dimostrare che è un problema finito o in forma chiusa
  - Dimostrare che la funzione caratteristica è calcolabile
  - Definire un algoritmo che decida S
  - Ridurre S ad un problema decidibile
  - Dimostrare che S ed S^c sono semidecidibili
- Per dimostrare che un problema S non è decidibile:
  - Teorema di Rice
  - Ridurre un problema non decidibile S' ad S
  - Dimostrare che S^c non è decidibile
  - Tecniche di diagonalizzazione
  - Dimostrare che la funzione caratteristica non è calcolabile (difficile!!)

Recap Tecniche di risoluzione dei quesiti

- Per dimostrare che un problema S è semidecidibile:
  - Dimostrare che S è dominio o immagine di una funzione (parziale o totale) calcolabile
  - Definire un algoritmo che semidecida S
  - Ridurre S ad un problema semidecidibile

- Per dimostrare che un problema S non è semidecidibile:
  - Dimostrare che S non è decidibile e che S^c è semidecidibile
  - Ridurre un problema indecidibile S' ad S
  - Dimostrare che la funzione semicaratteristica σ_S di S non è calcolabile (difficile!!)

Recap Tecniche di risoluzione dei quesiti

- Dato un problema S, come fare a decidere cosa dimostrare?
  - Lavorare *contemporaneamente* con S e con S^c!!
  - Pensare a un *algoritmo che decida* S e S^c → decidibile
  - Pensare a una funzione totale computabile di cui S (risp. S^c) sia l'immagine, ovvero una *enumerazione* per S (risp S^c) → semidecidibile
  - Pensare a un *algoritmo che semidecida* S → semidecidibile

    N.B.: condizione della forma $\forall x\ \varphi$ → non semidec. (di solito)
    condizione della forma $\exists x\ \varphi$ → semidec. (a volte) → tramite enumerazione diagonale

  - Niente di tutto ciò funziona? Allora iniziamo a pensare che sia indecidibile o non semidecidibile e proviamo con le relative tecniche

Recap Tecniche di risoluzione dei quesiti

- In generale, dato un problema S, si può sempre provare la seguente tecnica:
  - Riesco a dimostrare che è computabile la funzione caratteristica di S?

    Allora S è decidibile

    $$\chi_S(x) = \begin{cases} 1 \text{ se } x \in S \\ 0 \text{ se } x \notin S \end{cases}$$

  - Riesco a dimostrare che è computabile la funzione semicaratteristica di S?

    Allora S è semidecidibile

    $$\sigma_S(x) = \begin{cases} 1 \text{ se } x \in S \\ \perp \text{ se } x \notin S \end{cases}$$

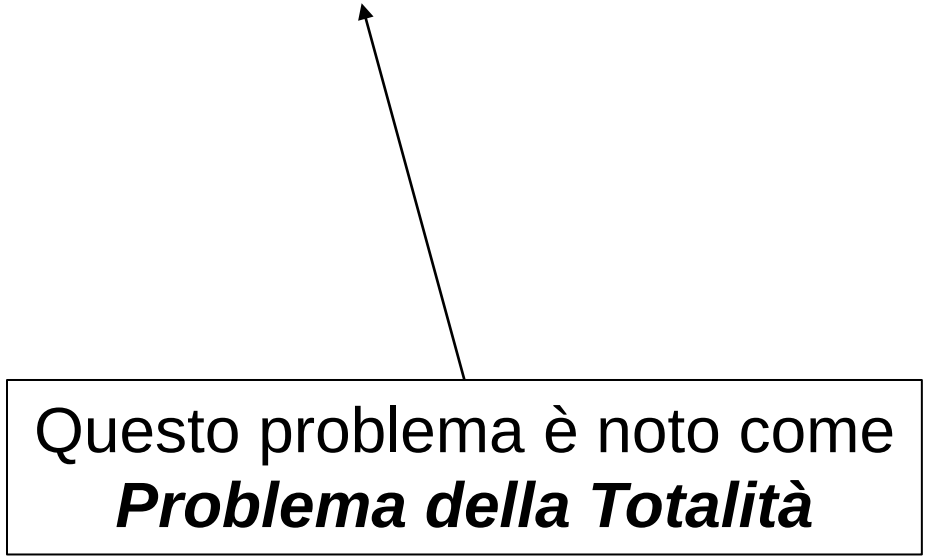
  - Mi rendo conto che una delle due non sembra affatto computabile? Allora passo a provare a dimostrare che S non è (semi)decidibile!

Recap Tecniche di risoluzione dei quesiti

- Oppure, meno formalmente:
  - Riesco a definire un algoritmo che decida S?
    Allora S è decidibile
  - Riesco a definire un algoritmo che semidecida S?
    Allora S è semidecidibile
  - Mi sembra che tali algoritmi non possano esistere?
    Allora provo a dimostrare che S non è (semi)decidibile

Es.0

Sia $Tot \coloneqq \{y \in \mathbb{N} \mid \forall x \; f_y(x) \neq \perp\}$ (funzioni totali)



Es.0

Sia $Tot \coloneqq \{y \in \mathbb{N} \mid \forall x \; f_y(x) \neq \perp\}$ (funzioni totali)

- Avete già visto che non è decidibile (si dimostra facilmente anche col teorema di Rice).
- E' semidecidibile?

Es.0

Sia $Tot := \{y \in \mathbb{N} \mid \forall x\ f_y(x) \neq \perp\}$ (funzioni totali)

- Avete già visto che non è decidibile (si dimostra facilmente anche col teorema di Rice).
- E' semidecidibile?

Intuizione:

- Consideriamo la sua funzione semicaratteristica, ovvero:

$$\sigma_{Tot}(y) = \begin{cases} 1 \text{ se } \forall x\left(f_y(x) \neq \perp\right) \\ \perp \text{ altrimenti} \end{cases}$$

- Servirebbero infinite computazioni per determinare che per un certo y si abbia $f_y(x) \neq \perp$ *per ogni* input x.
- Questo ci fa intuire che σ_{Tot} non sia calcolabile (e che dunque Tot non sia semidecibile), ma **non è una dimostrazione**!

Es.0

Sia $Tot \coloneqq \{y \in \mathbb{N} \mid \forall x\ f_y(x) \neq \perp\}$ (funzioni totali)

- E' semidecidibile? NO

Dimostrazione formale:

- Per assurdo supponiamo che Tot sia semidecidibile. Ma allora, essendo $Tot \neq \emptyset$, esiste una funzione calcolabile totale $enum: \mathbb{N} \to Tot$ che lo enumera.
- Definiamo $g(x) \coloneqq f_{enum(x)}(x) + 1$
- g è calcolabile totale, dunque $\exists j$ t.c. $g = f_j$, e $j \in Tot$
- Ma $enum$ enumera Tot, dunque $\exists \bar{x}$ t.c. $enum(\bar{x}) = j$
- Dunque $g(\bar{x}) = f_j(\bar{x}) = f_{enum(\bar{x})}(\bar{x})$
- Ma per definizione $g(\bar{x}) = f_{enum(\bar{x})}(\bar{x}) + 1$
- Assurdo

Es.1a

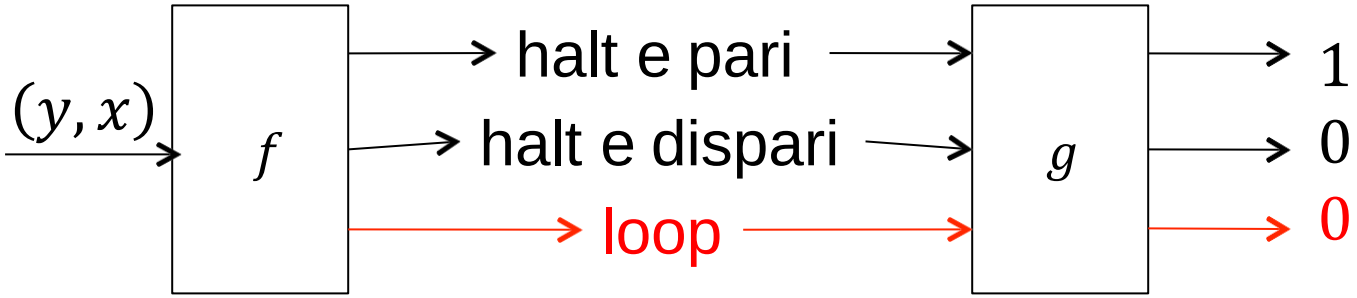
Stabilire se la seguente funzione è calcolabile:

$$g(y,x) = \begin{cases} 1 \text{ se } f_y(x) \text{ è pari} \\ 0 \text{ altrimenti} \end{cases}$$

Es.1a

Stabilire se la seguente funzione è calcolabile:

$$g(y,x) = \begin{cases} 1 \text{ se } f_y(x) \text{ è pari} \\ 0 \text{ altrimenti} \end{cases}$$



Ciò suggerisce che $g(y,x)$ non sia calcolabile, ma non è una dimostrazione…

Es.1a

Stabilire se la seguente funzione è calcolabile:

$$g(y,x) = \begin{cases} 1 \text{ se } f_y(x) \text{ è pari} \\ 0 \text{ altrimenti} \end{cases}$$

NO

Dimostriamolo col teorema di Rice:

- Fissiamo $k \in \mathbb{N}$ e consideriamo la funzione $h(y) \coloneqq g(y,k)$
- Abbiamo per definizione $h(y) = 1$ se $f_y(k)$ è pari, altrimenti 0.
- Dunque h è la funzione caratteristica di $S = \{y \in \mathbb{N} \mid f_y(k) \text{ è pari}\}$
- Siccome l'insieme $F = \{f \mid f(k) \text{ è pari}\}$ non è né vuoto né l'insieme di tutte le funzioni computabili, per Rice S non è decidibile, quindi la sua funziona caratteristica h non è calcolabile.
- Dunque g non è calcolabile, altrimenti lo sarebbe in particolare anche h.

Es.1b

Stabilire se la seguente funzione è calcolabile:

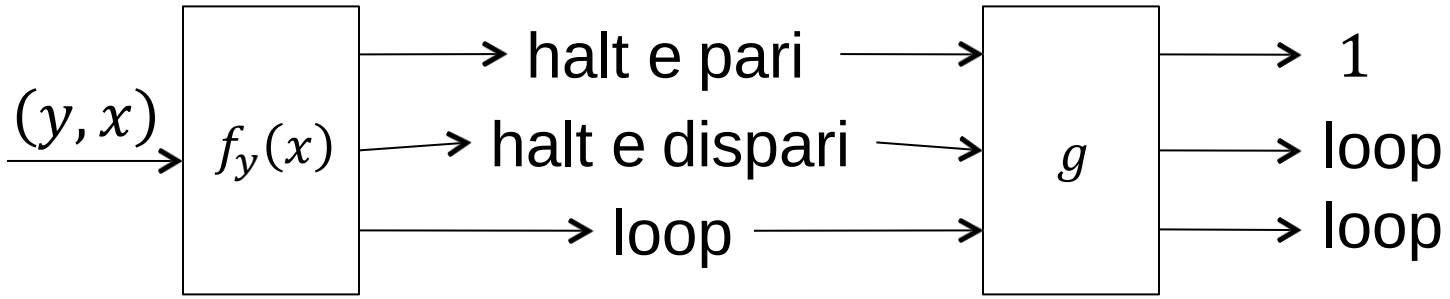
$$g(y,x) = \begin{cases} 1 \text{ se } f_y(x) \text{ è pari} \\ \perp \quad \text{altrimenti} \end{cases}$$

Es.1b

Stabilire se la seguente funzione è calcolabile:

$$g(y,x) = \begin{cases} 1 \text{ se } f_y(x) \text{ è pari} \\ \perp \quad \text{altrimenti} \end{cases}$$

SI'



Ciò non solo suggerisce che g sia calcolabile, ma lo dimostra: si tratta infatti di un procedimento algoritmico che calcola $g(y,x)$ per ogni y,x.

Dunque g è calcolabile per definizione.

Es.1c

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \forall x \ f_y(x) \text{ è pari} \\ 0 \ \text{ altrimenti} \end{cases}$$

Es.1c

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \forall x\ f_y(x) \text{ è pari} \\ 0 \text{ altrimenti} \end{cases}$$

NO

Dimostriamolo col teorema di Rice:

- Notiamo che g è la funzione caratteristica di

$$S = \{y \in \mathbb{N} \mid \forall x\ f_y(x) \text{ è pari}\}$$

- Siccome l'insieme $F = \{f \mid \forall x\ f(x) \text{ è pari}\}$ non è né vuoto né l'insieme di tutte le funzioni computabili, per Rice S non è decidibile, e quindi la sua funzione caratteristica g non è calcolabile.

Es.1d

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \forall x \ f_y(x) \text{ è pari} \\ \perp \text{ altrimenti} \end{cases}$$

Es.1d

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \forall x\ f_y(x) \text{ è pari} \\ \\ \perp \text{ altrimenti} \end{cases}$$

NO

Dimostriamolo per assurdo, sfruttando una riduzione:

- Notiamo che g è la funzione semicaratteristica dell'insieme

$$S \coloneqq \{y \in \mathbb{N} \mid \forall x\ f_y(x) \text{ è pari}\}$$

- Se g fosse calcolabile, S dovrebbe essere semidecidibile
- $Tot = \{y \in \mathbb{N} \mid \forall x\ f_y(x) \neq \perp\} \leq_r S$
- Abbiamo visto che Tot non è semidecidibile
- Dunque nemmeno S può esserlo

Es.1e

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \exists x \text{ t. c. } f_y(x) \text{ è pari} \\ 0 \text{ altrimenti} \end{cases}$$

Es.1e

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \exists x \text{ t.c. } f_y(x) \text{ è pari} \\ 0 \text{ altrimenti} \end{cases}$$

NO

Dimostriamolo col teorema di Rice:

- Notiamo che g è la funzione caratteristica di

$$S = \{y \in \mathbb{N} \mid \exists x \text{ t.c. } f_y(x) \text{ è pari}\}$$

- Siccome l'insieme $F = \{f \mid \exists x \text{ t.c. } f(x) \text{ è pari}\}$ non è né vuoto né l'insieme di tutte le funzioni computabili, per Rice S non è decidibile, e quindi la sua funziona caratteristica g non è calcolabile.

Es.1f

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \exists x \text{ t. c. } f_y(x) \text{ è pari} \\ \perp \text{ altrimenti} \end{cases}$$

Es.1f

Stabilire se la seguente funzione è calcolabile:

$$g(y) = \begin{cases} 1 \text{ se } \exists x \text{ t.c. } f_y(x) \text{ è pari} \\ \\ \perp \text{ altrimenti} \end{cases}$$

SI'

Dimostriamolo costruttivamente:

- Se è vero che $\exists x$ t.c. $f_y(x)$ è pari, allora si può scoprirlo in tempo finito tramite una tecnica di enumerazione diagonale, e dunque restituire 1.
- Se non è vero che $\exists x$ t.c. $f_y(x)$ è pari, nel cercare una tale x la macchina andrà in loop, ovvero "restituirà" $\perp$.

Es.2

Stabilire se la seguente funzione è calcolabile:

$$g(x,y,z) = \begin{cases} 1 \text{ se } f_x(y) = z \\ 0 \text{ altrimenti} \end{cases}$$

Es.2

Stabilire se la seguente funzione è calcolabile:

$$g(x,y,z) = \begin{cases} 1 \text{ se } f_x(y) = z \\ 0 \text{ altrimenti} \end{cases}$$

NO

Dimostriamolo tramite diagonalizzazione:

- Se g fosse calcolabile, dovrebbe esserlo anche $h: \mathbb{N} \to \mathbb{N}$ definita da

$$h(x) := g(x,x,0)$$

- Ma allora avremmo $h = f_i$ per un qualche $i \in \mathbb{N}$
- Dunque $f_i(i) = h(i) = g(i,i,0) = \begin{cases} 1 \text{ se } f_i(i) = 0 \\ 0 \text{ se } f_i(i) \neq 0 \end{cases}$
- Questa è una contraddizione

Es.3

Si consideri il seguente programma P, codificato in uno pseudo-codice ispirato al linguaggio C:

```
int x, y, z;
read (x, y, z);
while (x != y) {
    x = x - y;
    z = z + y;
}
write z;
```

Sia f la funzione calcolata da P.

Stabilire se la seguente funzione è calcolabile:

$$g(x,y,z) = \begin{cases} 1 \text{ se } f(x,y,z) \neq \perp \\ 0 \text{ altrimenti} \end{cases}$$

Es.3

Stabilire se la seguente funzione è calcolabile:

$$g(x,y,z) = \begin{cases} 1 \text{ se } f(x,y,z) \neq \perp \\ 0 \text{ altrimenti} \end{cases}$$

dove f è la funzione calcolata da P:

```
int x, y, z;
read (x, y, z);
while (x != y){
    x = x - y;
    z = z + y;
}
write z;
```

SI'

Dimostriamolo costruttivamente:

- Notiamo che il programma termina sse i dati forniti in input sono t.c. valga $\varphi(x,y) \coloneqq x = y \vee \Big((x > y > 0 \vee x < y < 0) \wedge \big(\exists k(k > 1 \wedge |x| = k * |y|)\big)\Big)$
- Quindi possiamo riscrivere la funzione g:

$$g(x,y,z) = \begin{cases} 1 \text{ se vale } \varphi(x,y) \\ 0 \text{ altrimenti} \end{cases}$$

- Tutte le condizioni di $\varphi(x,y)$ possono essere verificate algoritmicamente.

Es.4

- Consideriamo ora il problema di stabilire se la generica TM si arresti o meno sul generico input (***Halting Problem***).
- Studiamo questo problema in entrambe le sue formulazioni, ovvero quella per TM come calcolatori di funzioni e quella per TM come riconoscitori di linguaggi:
  - Halting Problem per TM come calcolatori di funzioni:

    «Dati una TM M ed un input x, è vero che $f_M(x) \neq \perp$?»
  - Halting Problem per TM come riconoscitori di linguaggi:

    «Dati una TM M ed un input x, è vero che $x \in L(M)$?»
- N.B.: le due soluzioni seguiranno esattamente gli stessi passaggi, ovvero la soluzione è la stessa, tradotta nelle due versioni.

Es.4a

Stabilire se il seguente problema è decidibile:

«Dati una TM M ed un input x, è vero che $f_M(x) \neq \perp$?»

Es.4a

Stabilire se il seguente problema è decidibile: NO

«Dati una TM M ed un input x, è vero che $f_M(x) \neq \perp$?»

Dimostriamolo col teorema di Rice:

- Consideriamo una particolarizzazione di questo problema, fissando l'input ad un certo valore $k \in \mathbb{N}$
- L'insieme che corrisponde alla particolarizzazione è $S = \{y \in \mathbb{N} \mid f_y(k) \neq \perp\}$
- Siccome l'insieme $F = \{f \mid f(k) \neq \perp\}$ non è né vuoto né l'insieme di tutte le funzioni computabili, per Rice S non è decidibile.
- Dunque non può essere decidibile nemmeno il problema di partenza, altrimenti lo sarebbe il suo caso particolare descritto da S.

Es.4a’

Stabilire se il seguente problema è decidibile:

«Dati una TM M ed un input x, è vero che $x \in L(M)$?»

Es.4a’

Stabilire se il seguente problema è decidibile: NO

«Dati una TM M ed un input x, è vero che $x \in L(M)$?»

Dimostriamolo col teorema di Rice:

- Consideriamo una particolarizzazione di questo problema, fissando l’input ad un certo valore $k \in \mathbb{N}$
- L’insieme che corrisponde alla particolarizzazione è $S = \{y \in \mathbb{N} \mid k \in L(M_y)\}$
- Siccome l’insieme di linguaggi riconoscibili $F = \{L \mid k \in L\}$ non è né vuoto né l’insieme di tutti i linguaggi riconoscibili, per Rice S non è decidibile.
- Dunque non può essere decidibile nemmeno il problema di partenza, altrimenti lo sarebbe il suo caso particolare descritto da S.

Es.4b

Stabilire se il seguente problema è semidecidibile:

«Dati una TM M ed un input x, è vero che $f_M(x) \neq \perp$?»

Es.4b

Stabilire se il seguente problema è semidecidibile: SI'

«Dati una TM M ed un input x, è vero che $f_M(x) \neq \perp$?»

Dimostriamolo sfruttando una condizione per RE:

- L'insieme S che rappresenta il problema è

$$S = \{(y, x) \in \mathbb{N} \times \mathbb{N} \mid f_y(x) \neq \perp\}$$

- S è semidecidibile in quanto la sua funzione semicaratteristica è computabile:

$$\sigma_S(y, x) = \begin{cases} 1 \text{ se } f_y(x) \neq \perp \\ \perp \text{ altrimenti} \end{cases}$$

Es.4b’

Stabilire se il seguente problema è semidecidibile:

«Dati una TM M ed un input x, è vero che $x \in L(M)$?»

Es.4b’

Stabilire se il seguente problema è semidecidibile: SI’

«Dati una TM M ed un input x, è vero che $x \in L(M)$?»

Dimostriamolo sfruttando una condizione per RE:

- L’insieme S che rappresenta il problema è

$$S = \{(y, x) \in \mathbb{N} \times \mathbb{N} \mid x \in L(M_y)\}$$

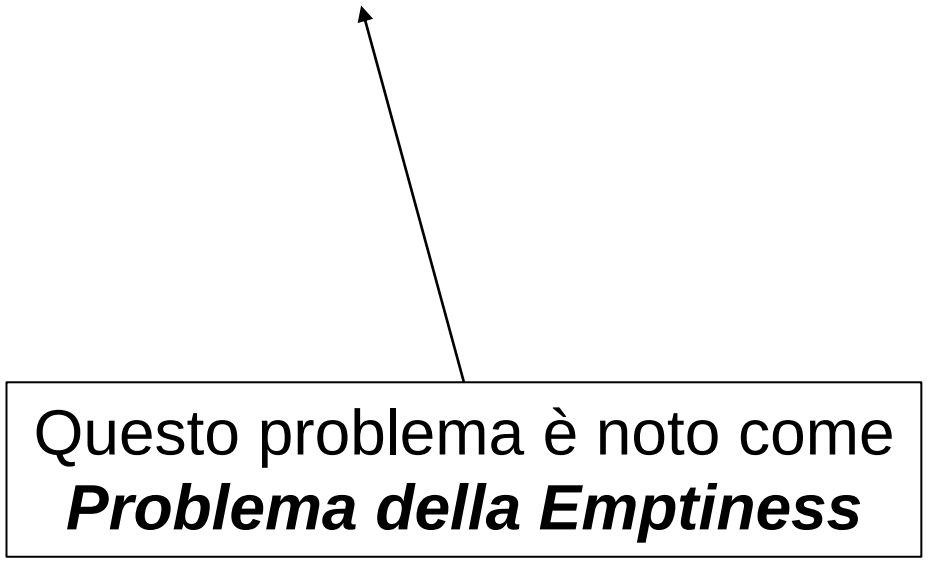
- S è semidecidibile in quanto la sua funzione semicaratteristica è computabile:

$$\sigma_S(y, x) = \begin{cases} 1 \text{ se } x \in L(M_y) \\ \perp \text{ altrimenti} \end{cases}$$

Es.5a

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che $L(M) = \emptyset$?»



Es.5a

Stabilire se il seguente problema è decidibile: NO

«Data una TM M, è vero che $L(M) = \emptyset$?»

Dimostriamolo col teorema di Rice:

- Se fosse decidibile, dovrebbe essere decidibile l'insieme

$$S = \{y \in \mathbb{N} \mid L(M_y) = \emptyset\}$$

- Siccome l'insieme di linguaggi riconoscibili $F = \{L \mid L = \emptyset\} = \{\emptyset\}$ non è né vuoto né l'insieme di tutti i linguaggi riconoscibili, per Rice S non è decidibile.

Es.5b

Stabilire se il seguente problema è semidecidibile:

«Data una TM M, è vero che $L(M) = \emptyset$?»

Es.5b

Stabilire se il seguente problema è semidecidibile: NO

«Data una TM M, è vero che $L(M) = \emptyset$?»

Dimostriamolo sfruttando il complementare:

- $S = \{y \in \mathbb{N} \mid L(M_y) = \emptyset\}$ non è decidibile.
- $S^c = \{y \in \mathbb{N} \mid \exists x\colon\ x \in L(M_y)\}$ è semidecidibile in quanto la sua funzione semicaratteristica è computabile tramite enumerazione diagonale:

$$\sigma_{S^c}(y) = \begin{cases} 1 \text{ se } \exists x\colon\ x \in L(M_y) \\ \perp \text{ altrimenti} \end{cases}$$

- Quindi S non può essere semidecidibile, altrimenti dovrebbe essere decidibile.

Es.5c

Stabilire se il seguente problema è decidibile:

«Data una TM M ed un insieme regolare $R \subseteq \mathbb{N}$,
è vero che $L(M) = R$?»

Es.5c

Stabilire se il seguente problema è decidibile: NO

«Data una TM M ed un insieme regolare $R \subseteq \mathbb{N}$,
è vero che $L(M) = R$?»

Dimostriamolo sfruttando una particolarizzazione:

- Se fosse decidibile, allora sarebbe decidibile anche il caso particolare con $R = \emptyset$, che è l'esercizio 5a.

Es.5d

Stabilire se il seguente problema è semidecidibile:

«Data una TM M ed un insieme regolare $R \subseteq \mathbb{N}$,
è vero che $L(M) = R$?»

Es.5d

Stabilire se il seguente problema è semidecidibile: NO

«Data una TM M ed un insieme regolare $R \subseteq \mathbb{N}$,
è vero che $L(M) = R$?»

Dimostriamolo sfruttando una particolarizzazione:

- Se fosse semidecidibile, allora sarebbe semidecidibile anche il caso particolare con $R = \emptyset$, che è l'esercizio 5b.

Es.5e

Stabilire se, data la TM M, il seguente problema è decidibile:

«E' vero che $L(M) = \emptyset$?»

Es.5e

Stabilire se, data la TM M, il seguente problema è decidibile:

«E' vero che $L(M) = \emptyset$?» SI'

E' un problema in forma chiusa!

N.B.: Il problema è decidibile, ma non è deciso:
la risposta dipenderà dalla TM data inizialmente.

Es.6

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata soddisfa φ?»

$$\varphi \coloneqq \forall x \Big((\neg(x \geq 0\ ) \rightarrow f(x) \neq 5) \wedge \Big(x \geq 0 \rightarrow \Big(f(x) > 37 \vee \big(\neg(f(x) = \bot) \rightarrow f(x) < 100\big)\Big)\Big)\Big)$$

Es.6

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata soddisfa φ?»

$$\varphi \coloneqq \forall x \left((\neg(x \geq 0\ ) \rightarrow f(x) \neq 5) \wedge \left(x \geq 0 \rightarrow \left(f(x) > 37 \vee \left(\neg(f(x) = \perp) \rightarrow f(x) < 100 \right) \right) \right) \right)$$

Osservazioni preliminari:

- L'insieme corrispondente al problema dato è $S = \{y \in \mathbb{N} \mid f_y \vDash \varphi\}$
- Si tratta dunque di un problema caratterizzato da una <u>proprietà della funzione calcolata</u> dalla generica TM, e quindi valgono le ipotesi di Rice.
- Tuttavia, prima di affrettarci a concludere che il problema non sia decidibile, occorre capire se effettivamente l'insieme $F = \{f \mid f \vDash \varphi\}$ delle funzioni calcolabili che validano φ sia diverso dal vuoto e dall'insieme di tutte le funzioni calcolabili…

Es.6

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata soddisfa φ?»

$$\varphi \coloneqq \forall x \left(\underbrace{(\neg(x \geq 0\ ) \rightarrow f(x) \neq 5)}_{\text{Sempre True}} \wedge \left(\underbrace{x \geq 0}_{\text{Sempre True}} \rightarrow \left(f(x) > 37 \vee \left(\neg(f(x) = \bot) \rightarrow f(x) < 100 \right) \right) \right) \right)$$

Es.6

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata soddisfa φ?»

$$\varphi \coloneqq \forall x \left((\neg(x \geq 0\ ) \rightarrow f(x) \neq 5) \wedge \left(x \geq 0 \rightarrow \left(f(x) > 37 \vee \left(\neg(f(x) = \bot) \rightarrow f(x) < 100 \right) \right) \right) \right)$$

$$\Vert\Vert$$

$$\forall x \left(f(x) > 37 \vee \left(\neg(f(x) = \bot) \rightarrow f(x) < 100 \right) \right)$$

Es.6

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata soddisfa φ?»

$$\varphi \coloneqq \forall x \Big((\neg(x \geq 0\,) \rightarrow f(x) \neq 5) \wedge \Big(x \geq 0 \rightarrow \Big(f(x) > 37 \vee \big(\neg(f(x) = \bot) \rightarrow f(x) < 100\big)\Big)\Big)\Big)$$

$$\Big|\Big|\Big|$$

$$\forall x \Big(f(x) > 37 \vee \underbrace{\big(\underbrace{\neg(f(x) = \bot)}_{\text{False}} \rightarrow f(x) < 100\big)}_{\text{True}}\Big)$$

Se $f(x) = \bot$

True

Es.6

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata soddisfa φ?»

$$\varphi \coloneqq \forall x \left(\left(\neg (x \geq 0\ ) \rightarrow f(x) \neq 5 \right) \wedge \left(x \geq 0 \rightarrow \left(f(x) > 37 \vee \left(\neg (f(x) = \bot) \rightarrow f(x) < 100 \right) \right) \right) \right)$$

$$\Vert\Vert$$

$$\forall x \left(f(x) > 37 \vee \underbrace{\left(\neg (f(x) = \bot) \right)}_{\text{True}} \rightarrow f(x) < 100 \right)$$

Se $f(x) \neq \bot$

$$\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad}_{\text{True}}$$

Es.6

Stabilire se il seguente problema è decidibile: SI'

«Data una TM M, è vero che la funzione calcolata soddisfa φ?»

$$\varphi \coloneqq \forall x \left((\neg(x \geq 0) \rightarrow f(x) \neq 5) \wedge \left(x \geq 0 \rightarrow \left(f(x) > 37 \vee \left(\neg(f(x) = \bot) \rightarrow f(x) < 100 \right) \right) \right) \right)$$

Conclusione:

- Dunque la proprietà è sempre vera, ovvero:

$$\{ f \mid f \vDash \varphi \} = \{\text{funzioni calcolabili}\}$$

- Quindi il problema è banalmente decidibile.
- N.B.: Il problema non è solo decidibile, ma è anche deciso: la risposta sarà sempre «Sì».

Es.7a

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata, laddove è definita, è costante?»

Es.7a

Stabilire se il seguente problema è decidibile: NO

«Data una TM M, è vero che la funzione calcolata, laddove è definita, è costante?»

Dimostriamolo col teorema di Rice:

- L'insieme corrispondente al problema è

$$S = \{y \in \mathbb{N} \mid f_y \text{ è costante nel suo dominio}\}$$

- Siccome l'insieme $F = \{f \mid f \text{ è costante nel suo dominio}\}$ non è né vuoto né l'insieme di tutte le funzioni computabili, per Rice S non è decidibile.

Es.7b

Stabilire se il seguente problema è semidecidibile:

«Data una TM M, è vero che la funzione calcolata, laddove è definita, è costante?»

Es.7b

Stabilire se il seguente problema è semidecidibile: NO

«Data una TM M, è vero che la funzione calcolata, laddove è definita, è costante?»

Dimostriamolo sfruttando il complementare:

- $S = \{y \mid f_y \text{ è costante nel suo dominio}\}$ non è decidibile.
- $S^c = \{y \mid f_y \text{ non è costante nel suo dominio}\}$ è semidecidibile in quanto la sua funzione semicaratteristica è computabile (tramite enumerazione diagonale):

$$\sigma_{S^c}(y) = \begin{cases} 1 \text{ se } \exists x, x' \left(f_y(x) \neq \perp \wedge f_y(x') \neq \perp \wedge f_y(x) \neq f_y(x')\right) \\ \perp \text{ altrimenti} \end{cases}$$

- Quindi S non può essere semidecidibile, altrimenti dovrebbe essere decidibile.

Es.8a

Stabilire se il seguente insieme è decidibile:

$$S = \{y \in \mathbb{N} \mid \forall x (f_y(x) \neq \perp \rightarrow f_y(x) = x^2)\}$$

Es.8a

Stabilire se il seguente insieme è decidibile: NO

$$S = \{y \in \mathbb{N} \mid \forall x(f_y(x) \neq \perp \rightarrow f_y(x) = x^2)\}$$

Applicazione diretta del teorema di Rice:

- Siccome l'insieme

$$F = \{f \mid \forall x(f(x) \neq \perp \rightarrow f(x) = x^2)\}$$

non è né vuoto né l'insieme di tutte le funzioni computabili, per Rice S non è decidibile.

Es.8b

Stabilire se il seguente insieme è semidecidibile:

$$S = \{y \in \mathbb{N} \mid \forall x(f_y(x) \neq \perp \rightarrow f_y(x) = x^2)\}$$

Es.8b

Stabilire se il seguente insieme è semidecidibile: NO

$$S = \{y \in \mathbb{N} \mid \forall x(f_y(x) \neq \perp \rightarrow f_y(x) = x^2)\}$$

Dimostriamolo sfruttando il complementare:

- S non è decidibile
- $S^c = \{y \mid \exists x(f_y(x) \neq \perp \wedge f_y(x) \neq x^2)\}$ è semidecidibile in quanto la sua funzione semicaratteristica è computabile (tramite enumerazione diagonale):

$$\sigma_{S^c}(y) = \begin{cases} 1 \text{ se } \exists x(f_y(x) \neq \perp \wedge f_y(x) \neq x^2) \\ \perp \text{ altrimenti} \end{cases}$$

- Quindi S non può essere semidecidibile, altrimenti dovrebbe essere decidibile.

Es.9a

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata è tale che valga $\varphi \coloneqq \forall x(x^2 + f(x)^2 + 1 = 0)$?»

Es.9a

Stabilire se il seguente problema è decidibile: SI'

«Data una TM M, è vero che la funzione calcolata è tale che valga $\varphi := \forall x(x^2 + f(x)^2 + 1 = 0)$?»

Dimostriamolo col teorema di Rice:

- L'insieme corrispondente al problema dato è $S = \{y \in \mathbb{N} \mid f_y \vDash \varphi\}$
- Quindi valgono le ipotesi di Rice, ma occorre capire se effettivamente l'insieme $F = \{f \mid f \vDash \varphi\}$ delle funzioni calcolabili che validano φ sia diverso dal vuoto e dall'insieme di tutte le funzioni calcolabili.
- E' facile osservare che φ è falsa per qualunque funzione $f: \mathbb{N} \to \mathbb{N}$, e dunque $F = \emptyset$. Segue che il problema è banalmente decidibile.
- N.B.: Il problema non è solo decidibile ma è anche deciso: la risposta sarà sempre «No».

Es.9b

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che la funzione calcolata è tale che valga $\varphi \coloneqq \forall x(f(x) = \perp \rightarrow x^2 + f(x)^2 + 1 = 0)$?»

Es.9b

Stabilire se il seguente problema è decidibile: NO

«Data una TM M, è vero che la funzione calcolata è tale che valga $\varphi := \forall x(f(x) = \perp \rightarrow x^2 + f(x)^2 + 1 = 0)$?»

Dimostriamolo col teorema di Rice:

- La proprietà φ non è più sempre falsa, ma vale per tutte e sole le funzioni totali: falsificando l'antecedente, l'implicazione risulta vera, anche se il conseguente di per sé è sempre falso.
- Dunque il problema corrisponde all'insieme $S = \{y \in \mathbb{N} \mid f_y \text{ funzione totale}\}$
- Sappiamo già che S non è decidibile per Rice.

Es.9c

Stabilire se il seguente problema è semidecidibile:

«Data una TM M, è vero che la funzione calcolata è tale che valga $\varphi \coloneqq \forall x(f(x) = \perp \rightarrow x^2 + f(x)^2 + 1 = 0)$?»

Es.9c

Stabilire se il seguente problema è semidecidibile: NO

«Data una TM M, è vero che la funzione calcolata è tale che valga $\varphi \coloneqq \forall x(f(x) = \perp \rightarrow x^2 + f(x)^2 + 1 = 0)$?»

Abbiamo già visto che l'insieme $S = \{y \in \mathbb{N} \mid f_y \text{ funzione totale}\}$ non è nemmeno semidecidibile!

Es.10

- Consideriamo ora il problema di stabilire se due programmi C dati calcolino o meno la stessa funzione (***Problema dell'Equivalenza***).
- Osservazione preliminare: dare due programmi C generici è equivalente a dare due TM generiche.
- Altra osservazione preliminare:
  - $f_y = f_{y'} \Leftrightarrow \forall x \left(f_y(x) = f_{y'}(x) \right)$ (stessa funzione)
  - $f_y \neq f_{y'} \Leftrightarrow \exists x \left(f_y(x) \neq f_{y'}(x) \right)$ (funzioni diverse)
- Vedremo poi cosa cambia assumendo di sapere a priori che entrambi i programmi dati terminano per ogni input, ovvero che calcolano funzioni totali.

Es.10a

Stabilire se il seguente problema è decidibile:

«Dati due programmi C P e P', è vero che calcolano la stessa funzione?»

Es.10a

Stabilire se il seguente problema è decidibile: NO

«Dati due programmi C P e P', è vero che calcolano la stessa funzione?»

Dimostriamolo col teorema di Rice:

- Fissiamo uno dei due programmi, ad esempio P. Il problema diventa: «Dato un programma P', è vero che la funzione calcolata da P' coincide con la funzione f calcolata da P?»
- Quest'ultimo problema corrisponde all'insieme

$$S = \{y \in \mathbb{N} \mid f_y = f\}$$

- S non è decidibile per Rice, dunque nemmeno il problema di partenza può esserlo.

Es.10b

Stabilire se il seguente problema è decidibile:

«Dati due programmi C P e P', è vero che calcolano funzioni diverse?»

Es.10b

Stabilire se il seguente problema è decidibile: NO

«Dati due programmi C P e P', è vero che calcolano funzioni diverse?»

Dimostriamolo sfruttando il complementare:

- Il complementare di un problema decidibile è anch'esso decidibile.
- Questo problema è chiaramente il complementare del 10a, che abbiamo dimostrato non essere decidibile.
- Dunque questo problema non può essere decidibile, altrimenti dovrebbe esserlo anche il 10a.
- N.B.: si poteva dimostrarlo anche direttamente con Rice.

Es.10c

Stabilire se il seguente problema è semidecidibile:

«Dati due programmi C P e P', è vero che calcolano la stessa funzione?»

Es.10c

Stabilire se il seguente problema è semidecidibile: NO

«Dati due programmi C P e P', è vero che calcolano la stessa funzione?» (Problema1)

Dimostriamolo sfruttando una riduzione:

- Fissiamo P' a un programma che calcola la funzione indefinita ovunque. Il problema diventa di determinare se una generica TM calcoli la funzione indefinita ovunque (Problema2, che è il problema della emptiness per le TM come calcolatori di funzioni).
- Se il Problema1 fosse semidecidibile, anche il Problema2 lo sarebbe.
- Il Problema2 non è decidibile per il teorema di Rice.
- Il complementare del Problema2 è semidecidibile tramite enumerazione diagonale (se c'è un valore per cui la funzione calcolata è definita, prima o poi lo scopro).
- Dunque il Problema2 non può essere semidecidibile, e quindi neanche il Problema1.

Es.10d

Stabilire se il seguente problema è semidecidibile:

«Dati due programmi C P e P', è vero che calcolano funzioni diverse?»

Es.10d

Stabilire se il seguente problema è semidecidibile: NO(!!)

«Dati due programmi C P e P', è vero che calcolano funzioni diverse?»

La dimostrazione è complessa.
Ci limitiamo ad una argomentazione intuitiva:

- L'insieme S che rappresenta il problema è $S = \{(y, y') \mid \exists x(f_y(x) \neq f_{y'}(x))\}$
- Notiamo che la funzione semicaratteristica di S non sembra computabile:

$$\sigma_S(y, y') = \begin{cases} 1 \text{ se } \exists x \left(f_y(x) \neq f_{y'}(x)\right) \\ \perp \text{ altrimenti} \end{cases}$$

N.B.: l'enumerazione diagonale non funziona: due funzioni potrebbero differire solo in $\bar{x}$ con $f_y(\bar{x}) = \perp$, $f_{y'}(\bar{x}) \neq \perp$

N.B.: QUESTO RAGIONAMENTO NON E' UNA DIMOSTRAZIONE

Es.10e

Stabilire se il seguente problema è decidibile:

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano la stessa funzione?»

Es.10e

Stabilire se il seguente problema è decidibile:

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano la stessa funzione?»

Dimostriamolo col teorema di Rice (??)

- Fissiamo uno dei due programmi, ad esempio P. Il problema diventa: «è vero che P' calcola la funzione totale f calcolata da P?»
- Quest'ultimo problema corrisponde all'insieme

$$S = \{y \in \mathbb{N} \mid f_y = f\}$$

- Concludiamo come al solito che non è decidibile.

N.B.: QUESTO RAGIONAMENTO E' SBAGLIATO: Non stiamo lavorando con una codifica (goedelizzazione) dell'insieme di tutte le TM, dunque non siamo sotto le ipotesi di Rice!

Es.10e

Stabilire se il seguente problema è decidibile: NO(!!)

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano la stessa funzione?»

La dimostrazione è complessa.
Ci limitiamo ad una argomentazione intuitiva:

- Un algoritmo non può essere in grado di decidere il problema, ovvero di determinare se P e P' calcolano la stessa funzione o meno: infatti, se effettivamente calcolano la stessa funzione, per poterlo scoprire si dovrebbero fare infinite computazioni, una per ogni intero, e verificare ogni volta che i risultati coincidano.

N.B.: QUESTO RAGIONAMENTO NON E' UNA DIMOSTRAZIONE

Es.10f

Stabilire se il seguente problema è decidibile:

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano funzioni diverse?»

Es.10f

Stabilire se il seguente problema è decidibile: NO

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano funzioni diverse?»

Dimostriamolo sfruttando il punto precedente:

- Il complementare di un problema decidibile è anch'esso decidibile.
- Questo problema è il complementare del 10e (passaggio delicato, attenzione!), che non è decidibile.
- Dunque questo problema non può essere decidibile, altrimenti dovrebbe esserlo anche il 10e.

Es.10f

Stabilire se il seguente problema è decidibile: NO

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano funzioni diverse?»

Diamo anche un'argomentazione intuitiva:

- Un algoritmo non può essere in grado di decidere il problema, ovvero di determinare se P e P' calcolano funzioni diverse o meno.
- Se calcolano funzioni diverse, lo si può effettivamente scoprire con un numero finito di computazioni (sapendo che sicuramente P e P' terminano su ogni input) tramite enumerazione diagonale.
- Ma se calcolano la stessa funzione, per poterlo scoprire si dovrebbero fare infinite computazioni.

N.B.: QUESTO RAGIONAMENTO NON E' UNA DIMOSTRAZIONE

Es.10g

Stabilire se il seguente problema è semidecidibile:

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano funzioni diverse?»

Es.10g

Stabilire se il seguente problema è semidecidibile: SI'

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano funzioni diverse?»

Dimostriamolo in modo diretto:

- Esiste un algoritmo che semidecide il problema.
- Infatti, se effettivamente i due programmi in input calcolano funzioni diverse, allora $\exists x \left(f_y(x) \neq f_{y'}(x) \right)$ e, siccome la terminazione è sempre garantita su ogni input, lo si può scoprire con un numero finito di computazioni (tramite enumerazione diagonale).
- N.B.: Se invece i due programmi calcolano la stessa funzione, il nostro algoritmo andrà in loop infinito.

Es.10h

Stabilire se il seguente problema è semidecidibile:

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano la stessa funzione?»

Es.10h

Stabilire se il seguente problema è semidecidibile: NO

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano la stessa funzione?»

Dimostriamolo sfruttando i punti precedenti:

- Se un problema non è decidibile, ed il suo complementare è semidecidibile, allora il problema è indecidibile.
- Questo problema è il complementare del 10g, che è semidecidibile.
- Ma questo problema non è decidibile (10e).

Es.10h

Stabilire se il seguente problema è semidecidibile: NO

«Dati due programmi C P e P' di cui è noto che terminano per ogni valore di input, è vero che calcolano la stessa funzione?»

Diamo anche un'argomentazione intuitiva:

- Un algoritmo non può essere in grado di determinare se P e P' calcolino la stessa funzione o meno: infatti, se effettivamente calcolano la stessa funzione, per poterlo scoprire si dovrebbero fare infinite computazioni, una per ogni input, e verificare ogni volta che i risultati coincidano (terminano tutte, ma sono infinite).

N.B.: QUESTO RAGIONAMENTO NON E' UNA DIMOSTRAZIONE

Es.11a

Stabilire se il seguente problema è decidibile:

«Dato un FSA A, è vero che $L(A) = \emptyset$?»

Es.11a

Stabilire se il seguente problema è decidibile: SI'

«Dato un FSA A, è vero che $L(A) = \emptyset$?»

Definiamo un algoritmo che decida il problema:

- Sappiamo che $L(A) = \emptyset \Leftrightarrow \ \forall w \left(|w| < |A| \rightarrow w \notin L(A) \right)$
- Dunque basta controllare l'accettazione di un numero finito di parole: se l'automa non ne accetta nessuna, l'algoritmo restituisce SI'; se l'automa ne accetta almeno una, l'algoritmo restituisce NO.

Es.11b

Stabilire se il seguente problema è decidibile:

«Dati due FSA A_1 e A_2, è vero che $L(A_1) = L(A_2)$?»

Es.11b

Stabilire se il seguente problema è decidibile: SI'

«Dati due FSA A_1 e A_2, è vero che $L(A_1) = L(A_2)$?»

Definiamo un algoritmo che decida il problema:

- Abbiamo che $L(A_1) = L(A_2) \Leftrightarrow L(A_1) \cap L(A_2)^c = \emptyset \wedge L(A_1)^c \cap L(A_2) = \emptyset$
- Sappiamo che esistono algoritmi che costruiscono il complemento e l'intersezione di FSA, e abbiamo appena visto un algoritmo che verifica se il linguaggio riconosciuto da un FSA è vuoto o meno.
- E' dunque chiaro come ottenere un algoritmo che decida il problema in questione.

Es.12a

Stabilire se, date le due grammatiche G_1 e G_2,
il seguente problema è decidibile:

«G_1 è equivalente a G_2?»

G_1:	G_2:
S→ADaCD|DAbDC	S→aADCD|DbADC
DC→CD|AA	DC→CD|AA
A→a|SSAD	A→a|SSAD|Ɛ
C→ADC|C	C→ADC|C
SA→AS|Ɛ	SA→AS|Ɛ

Es.12a

Stabilire se, date le due grammatiche G_1 e G_2, il seguente problema è decidibile:

«G_1 è equivalente a G_2?»

SI'

G_1:

S→ADaCD|DAbDC

DC→CD|AA

A→a|SSAD

C→ADC|C

SA→AS|ε

G_2:

S→aADCD|DbADC

DC→CD|AA

A→a|SSAD|ε

C→ADC|C

SA→AS|ε

E' un problema in forma chiusa!

N.B.: Il problema è decidibile ma non è deciso: la risposta dipende dalle due grammatiche date.

Es.12b

Stabilire se il seguente problema è decidibile:

«Date due generiche grammatiche G_1, G_2, esse sono equivalenti?»

Es.12b

Stabilire se il seguente problema è decidibile: NO

«Date due generiche grammatiche G_1, G_2, esse sono equivalenti?»

Dimostriamolo sfruttando la Turing-completezza:

- Sappiamo che le grammatiche generali sono un formalismo Turing-completo.
- Dunque il problema dato equivale al seguente problema: «determinare se i linguaggi riconosciuti da due generiche TM siano lo stesso linguaggio». Questo non è altro che il problema dell'equivalenza, nella versione per le TM come riconoscitori di linguaggi.
- Il problema dell'equivalenza non è decidibile per Rice (vedi Es.10a).

Es.12c

Stabilire se il seguente problema è semidecidibile:

«Date due generiche grammatiche G_1, G_2, esse sono equivalenti?» (problema P_1)

Es.12c

Stabilire se il seguente problema è semidecidibile: NO

«Date due generiche grammatiche G_1, G_2, esse sono equivalenti?» (problema P_1)

Dimostriamolo sfruttando la Turing-completezza:

- Come prima, data la Turing-completezza delle grammatiche generali, ci possiamo ricondurre al problema della equivalenza per TM.
- Ma abbiamo visto che il problema dell'equivalenza non è nemmeno semidecidibile (vedi Es.10c).

Es.12d

Stabilire se il seguente problema è decidibile:

«Date due generiche grammatiche regolari G_1, G_2, esse sono equivalenti?»

Es.12d

Stabilire se il seguente problema è decidibile: SI'

«Date due generiche grammatiche regolari G_1, G_2, esse sono equivalenti?»

Definiamo un algoritmo che decida il problema:

- Sappiamo che esistono algoritmi che, data una grammatica regolare G, costruiscono un FSA che accetta il linguaggio generato da G.
- Dunque il problema in questione è riducibile al problema di determinare se due generici FSA accettano lo stesso linguaggio.
- Abbiamo visto che quest'ultimo problema è decidibile (Es.11b).
- E' dunque chiaro come ottenere un algoritmo che decida il problema in questione.

Es.13a

Stabilire se il seguente problema è decidibile:

«Data una TM M e due configurazioni c_1, c_2 di M, esse sono in relazione di transizione a un passo (ovvero $c_1 \vdash_M c_2$)?»

Es.13a

Stabilire se il seguente problema è decidibile: SI'

«Data una TM M e due configurazioni c_1, c_2 di M, esse sono in relazione di transizione a un passo (ovvero $c_1 \vdash_M c_2$)?»

Definiamo un algoritmo che decida il problema:

- Prendiamo in input M, c_1, c_2
- Controlliamo che tra tutte le transizioni di M abilitate nello stato c_1, ce ne sia una che porta nella stato c_2
- Tali transizioni sono in numero finito, quindi è possibile determinare in un numero finito di passi sia se $c_1 \vdash c_2$, sia se $c_1 \nvdash c_2$
- Dunque il problema in questione è decidibile.

Es.13b

Stabilire se il seguente problema è decidibile:

«Data una TM M e due configurazioni c_1, c_2 di M, esse sono in relazione di transizione a più passi (ovvero $c_1 \vdash_M^* c_2$)?»

Es.13b

Stabilire se il seguente problema è decidibile: NO

«Data una TM M e due configurazioni c_1, c_2 di M, esse sono in relazione di transizione a più passi (ovvero $c_1 \vdash_M^* c_2$)?»

Dimostriamolo sfruttando una riduzione:

- Mostriamo che l'Halting Problem è riducibile a questo problema (slide successiva. Ci riferiremo a questo problema come Problema1).
- Siccome l'Halting problem non è decidibile (Es.3a), potremo concludere che nemmeno questo problema lo possa essere.

Es.13b

Riduzione dell'Halting Problem al Problema1

- Abbiamo una TM M e un input x (istanza dell'Halting Problem: «$x \in L(M)$?»)
- Modifichiamo M secondo la seguente procedura algoritmica:
  - Aggiungiamo uno stato finale e rendiamo non più finali tutti gli stati finali di M.
  - Modifichiamo la funzione di transizione di M in modo che quando arriva in uno stato ex-finale inizi a cancellare i nastri e a portare la testina di input alla fine della porzione non blank del nastro di input, per poi portarsi nel nuovo stato finale solo quando tutti i nastri sono vuoti e la testina di input punta sul primo blank
- Otteniamo così una TM M' che simula M ma ha una sola configurazione finale per ogni input x, chiamiamola $c_{fin}(x)$: così si ha che

$$x \in L(M) \Leftrightarrow c_0(x) \vdash_{M'}^{*} c_{fin}(x)$$

- In questo modo abbiamo ridotto l'Halting Problem, ovvero il problema di determinare se $x \in L(M)$, al problema di determinare se $c_0(x) \vdash_{M'}^{*} c_{fin}(x)$, che è un'istanza del Problema1.

Es.13c

Stabilire se il seguente problema è semidecidibile:

«Data una TM M e due configurazioni c_1, c_2 di M, esse sono in relazione di transizione a più passi (ovvero $c_1 \vdash_M^* c_2$)?»

Es.13c

Stabilire se il seguente problema è semidecidibile: SI'

«Data una TM M e due configurazioni c_1, c_2 di M, esse sono in relazione di transizione a più passi (ovvero $c_1 \vdash_M^* c_2$)?»

Definiamo un algoritmo che semidecida il problema:

- Prende in input M, c_1, c_2
- Esplora in ampiezza l'albero di computazioni (o la computazione se M è deterministica) di M che parte da c_1
- Se è vero che $c_1 \vdash_M^* c_2$, allora lo scoprirà in un numero finito di passi e dunque restituirà SI'.
- Altrimenti potrebbe andare in loop o esaurire l'albero delle computazioni, nel qual caso mandiamo comunque forzatamente in loop.

Es.14a

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che $\epsilon \in L(M)$?»

Es.14a

Stabilire se il seguente problema è decidibile: NO

«Data una TM M, è vero che $\epsilon \in L(M)$?»

Dimostriamolo col teorema di Rice:

- Assumiamo che 0 sia la codifica di ϵ
- Allora la condizione da verificare è $0 \in L(M)$
- Se il problema fosse decidibile, dovrebbe essere decidibile l'insieme

$$S = \{y \in \mathbb{N} \mid 0 \in L(M_y)\}$$

- Siccome l'insieme di linguaggi $F = \{L \mid 0 \in L\}$ non è né vuoto né l'insieme di tutti i linguaggi riconoscibili, per Rice S non è decidibile.

Es.14b

Stabilire se il seguente problema è semidecidibile:

«Data una TM M, è vero che $\epsilon \in L(M)$?»

Es.14b

Stabilire se il seguente problema è semidecidibile: SI'

«Data una TM M, è vero che $\epsilon \in L(M)$?»

Dimostriamolo sfruttando una condizione per RE:

- L'insieme S che rappresenta il problema è

$$S = \{y \in \mathbb{N} \mid 0 \in L(M_y)\}$$

- S è semidecidibile in quanto la sua funzione semicaratteristica è computabile:

$$\sigma_S(y) = \begin{cases} 1 \text{ se } 0 \in L(M_y) \\ \perp \text{ altrimenti} \end{cases}$$

Es.15a

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che, se fatta partire con input ϵ, scrive tre 1 di fila in output?»

Es.15a

Stabilire se il seguente problema è decidibile: NO

«Data una TM M, è vero che, se fatta partire con input ϵ, scrive tre 1 di fila in output?»

Dimostriamolo col teorema di Rice:

- Assumiamo che 0 sia la codifica di ϵ
- Allora l'insieme corrispondente al problema è:

$$S = \{y \in \mathbb{N} \mid f_y(0) \text{ contiene tre 1 di fila}\}$$

- Siccome l'insieme $F = \{f \mid f(0) \text{ contiene tre 1 di fila}\}$ non è né vuoto né l'insieme di tutte le funzioni computabili, per Rice S non è decidibile.

Es.15b

Stabilire se il seguente problema è semidecidibile:

«Data una TM M, è vero che, se fatta partire con input ϵ, scrive tre 1 di fila in output?»

Es.15b

Stabilire se il seguente problema è semidecidibile: SI'

«Data una TM M, è vero che, se fatta partire con input ϵ, scrive tre 1 di fila in output?»

Dimostriamolo sfruttando una condizione per RE:

- L'insieme S che rappresenta il problema è

$$S = \{y \in \mathbb{N} \mid f_y(0) \text{ contiene tre 1 di fila}\}$$

- S è semidecidibile in quanto la sua funzione semicaratteristica è computabile:

$$\sigma_S(y) = \begin{cases} 1 \text{ se } f_y(0) \text{ contiene tre 1 di fila} \\ \perp \text{ altrimenti} \end{cases}$$

Es.16

Stabilire se il seguente problema è decidibile:

«Data una TM M, è vero che M termina la sua esecuzione in meno di 100 passi per qualche input?»

Es.16

Stabilire se il seguente problema è decidibile: NO(??)

«Data una TM M, è vero che M termina la sua esecuzione in meno di 100 passi per qualche input?»

Verrebbe in mente di applicare il teorema di Rice: (??)

- L'insieme S che rappresenta il problema è

$$S = \{y \in \mathbb{N} \mid \exists x(f_y(x) \text{ è calcolata in meno di } 100 \text{ passi})\}$$

- Ma $F = \{f \mid \exists x(f(x) \ldots)\}$ non è un insieme di funzioni!!
- Dunque S non è un insieme di indici di TM che calcolano funzioni con certe proprietà: la proprietà in questione non è una proprietà della funzione calcolata, ma una proprietà strutturale delle macchine.
- In conclusione non possiamo applicare il teorema di Rice.

Es.16

Stabilire se il seguente problema è decidibile: SI'!

«Data una TM M, è vero che M termina la sua esecuzione in meno di 100 passi per qualche input?»

Definiamo un algoritmo che decide S:

- Prendiamo in ingresso una TM e verifichiamo se per un qualche input di lunghezza minore di 100 essa termina la sua esecuzione entro 99 passi.
- Gli input da considerare sono in numero finito, quindi la verifica avviene in un numero finito di passi.
- Non serve controllare gli input più lunghi: in 99 mosse la macchina non può leggere 100 o più simboli.

Es.17

Si stabilisca se ciascuno dei seguenti insiemi è ricorsivo:

$S_1 = \{i \mid i \text{ è un numero primo}\}$

$S_2 = \{i \mid i \text{ è un numero primo e } i < 100\}$

$S_3 = \{i \mid i \text{ è l'indice di una TM che stabilisce se l'input è primo}\}$

$S_4 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM che stab. se l'input è primo}\}$

$S_5 = \{i \mid i \text{ è l'indice di una TM con un numero dispari di stati}\}$

$S_6 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM con un numero dispari di stati}\}$

Es.17

Si stabilisca se ciascuno dei seguenti insiemi è ricorsivo:

$S_1 = \{i \mid i \text{ è un numero primo}\}$

$\boldsymbol{S_2 = \{i \mid i \text{ è un numero primo e } i < 100\}}$

$S_3 = \{i \mid i \text{ è l'indice di una TM che stabilisce se l'input è primo}\}$

$\boldsymbol{S_4 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM che stab. se l'input è primo}\}}$

$S_5 = \{i \mid i \text{ è l'indice di una TM con un numero dispari di stati}\}$

$\boldsymbol{S_6 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM con un \# dispari di stati}\}}$

Soluzione:

- S_2, S_4, S_6 sono ricorsivi in quanto finiti.

Es.17

Si stabilisca se ciascuno dei seguenti insiemi è ricorsivo:

$\boldsymbol{S_1 = \{i \mid i \text{ è } \textbf{un numero primo}\}}$

$S_2 = \{i \mid i \text{ è un numero primo e } i < 100\}$

$S_3 = \{i \mid i \text{ è l'indice di una TM che stabilisce se l'input è primo}\}$

$S_4 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM che stab. se l'input è primo}\}$

$S_5 = \{i \mid i \text{ è l'indice di una TM con un numero dispari di stati}\}$

$S_6 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM con un numero dispari di stati}\}$

Soluzione:

- S_1 è ricorsivo in quanto la sua funzione caratteristica è computabile (deve stabilire se un numero è primo, il che è decidibile).

Es.17

Si stabilisca se ciascuno dei seguenti insiemi è ricorsivo:

$S_1 = \{i \mid i \text{ è un numero primo}\}$

$S_2 = \{i \mid i \text{ è un numero primo e } i < 100\}$

$\boldsymbol{S_3 = \{i \mid i \text{ è l'indice di una TM che stabilisce se l'input è primo}\}}$

$S_4 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM che stab. se l'input è primo}\}$

$S_5 = \{i \mid i \text{ è l'indice di una TM con un numero dispari di stati}\}$

$S_6 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM con un numero dispari di stati}\}$

Soluzione:

- S_3 non è ricorsivo per il teorema di Rice: l'insieme $F := \{f\}$, dove f è la funzione che stabilisce se il suo argomento sia primo o meno, non è né vuoto né l'insieme delle funzioni computabili, e si ha $S_3 = \{i \mid f_i \in F\}$.

Es.17

Si stabilisca se ciascuno dei seguenti insiemi è ricorsivo:

$S_1 = \{i \mid i \text{ è un numero primo}\}$

$S_2 = \{i \mid i \text{ è un numero primo e } i < 100\}$

$S_3 = \{i \mid i \text{ è l'indice di una TM che stabilisce se l'input è primo}\}$

$S_4 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM che stab. se l'input è primo}\}$

$\boldsymbol{S_5 = \{i \mid i \text{ è l'indice di una TM con un numero dispari di stati}\}}$

$S_6 = \{i \mid i < 100 \text{ e } i \text{ è l'indice di una TM con un numero dispari di stati}\}$

Soluzione:

- S_5 è ricorsivo: per ogni i basta infatti ispezionare il diagramma degli stati ricavato dall'enumerazione di Goedel per stabilire se il numero di stati di M_i sia pari o dispari.

Es.18a

Siano dati due DPDA A_1 e A_2. E' ricorsivo il linguaggio $L(A_1) \cap L(A_2)$?

Es.18a

Siano dati due DPDA A_1 e A_2. E' ricorsivo il linguaggio $L(A_1) \cap L(A_2)$?

Soluzione:

SI': data una generica stringa x, si può determinare algoritmicamente se x appartenga o meno a $L(A_1) \cap L(A_2)$. Basta infatti verificare separatamente l'appartenenza di x a $L(A_1)$ e a $L(A_2)$, e ciascuno di questi due test richiede tempo finito in quanto possiamo assumere che ciascun DPDA sia aciclico (se non fosse aciclico, lo si potrebbe comunque trasformare in un equivalente automa aciclico in modo algoritmico).

Es.18b

Siano dati due DPDA A_1 e A_2 ed una TM M. E' decidibile se M calcola $L(A_1) \cap L(A_2)$?

Es.18b

Siano dati due DPDA A_1 e A_2 ed una TM M. E' decidibile se M calcola $L(A_1) \cap L(A_2)$?

Soluzione:

NO, per il teorema di Rice: fissati A_1 e A_2, riconoscere il linguaggio $L(A_1) \cap L(A_2)$ è una proprietà non banale del linguaggio riconosciuto da una TM M.

Es.19

Sia m il numero di Goedel di una TM M che calcola la funzione $f(x)$. Si stabilisca se ciascuna delle seguenti funzioni è calcolabile:

$$g_1(x) = f(m)$$

$$g_2(x) = f(x)$$

$$g_3(x) = \begin{cases} f(x) - 9 & \text{se } f(x) > 10, \\ \perp & \text{altrimenti} \end{cases}$$

$$g_4(x) = \begin{cases} f_x(x) & \text{se } x = m, \\ 1 & \text{altrimenti} \end{cases}$$

Es.19

Sia m il numero di Goedel di una TM M che calcola la funzione $f(x)$. Si stabilisca se ciascuna delle seguenti funzioni è calcolabile:

$$g_1(x) = f(m)$$
$$g_2(x) = f(x)$$
$$g_3(x) = f(x) - 9 \text{ se } f(x) > 10, \quad \perp \text{ altrimenti}$$
$$g_4(x) = f_x(x) \text{ se } x = m, \quad 1 \text{ altrimenti}$$

Soluzione:

Sono tutte funzioni computabili. Infatti:

- g_1 è costante, in quanto $f(m)$ è uno specifico valore.

Es.19

Sia m il numero di Goedel di una TM M che calcola la funzione $f(x)$. Si stabilisca se ciascuna delle seguenti funzioni è calcolabile:

$$g_1(x) = f(m)$$
$$g_2(x) = f(x)$$
$$g_3(x) = f(x) - 9 \text{ se } f(x) > 10, \qquad \perp \text{ altrimenti}$$
$$g_4(x) = f_x(x) \text{ se } x = m, \qquad 1 \text{ altrimenti}$$

Soluzione:

Sono tutte funzioni computabili. Infatti:

- g_2 è computabile per definizione, in quanto la funzione f è calcolata dalla TM M.

Es.19

Sia m il numero di Goedel di una TM M che calcola la funzione $f(x)$. Si stabilisca se ciascuna delle seguenti funzioni è calcolabile:

$$g_1(x) = f(m)$$
$$g_2(x) = f(x)$$
$$g_3(x) = f(x) - 9 \text{ se } f(x) > 10, \qquad \perp \text{ altrimenti}$$
$$g_4(x) = f_x(x) \text{ se } x = m, \qquad 1 \text{ altrimenti}$$

Soluzione:

Sono tutte funzioni computabili. Infatti:

- Per calcolare g_3, si lancia la computazione di M su input generico x. Se termina, si controlla il risultato: se è > 10, se ne sottrae 9, altrimenti si entra in un loop infinito. Se non termina, il valore di g_3 è $\perp$, compatibilmente con la definizione.

Es.19

Sia m il numero di Goedel di una TM M che calcola la funzione $f(x)$. Si stabilisca se ciascuna delle seguenti funzioni è calcolabile:

$$g_1(x) = f(m)$$
$$g_2(x) = f(x)$$
$$g_3(x) = f(x) - 9 \text{ se } f(x) > 10, \qquad \perp \text{ altrimenti}$$
$$g_4(x) = f_x(x) \text{ se } x = m, \qquad 1 \text{ altrimenti}$$

Soluzione:

Sono tutte funzioni computabili. Infatti:

- Per calcolare g_4, si confronta x con m. Se $x = m$, allora si restituisce $f_x(x) = f_m(m) = f(m)$, che è calcolabile per definizione. Altrimenti, si restituisce 1.

Es.20a

Un professore ha costruito un programma che riceve in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisce in uscita la versione ordinata alfabeticamente dello stesso array.

a) E' decidibile stabilire se il programma costruito dal professore sia corretto rispetto alle specifiche fornite?

Es.20a

Un professore ha costruito un programma che riceve in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisce in uscita la versione ordinata alfabeticamente dello stesso array.

a) E' decidibile stabilire se il programma costruito dal professore sia corretto rispetto alle specifiche fornite?

Soluzione:

SI', è una domanda chiusa: si tratta di un singolo programma, quindi la risposta non dipende da alcun parametro.

N.B.: Il problema è decidibile ma non è deciso:
la risposta dipende dal programma costruito.

Es.20b

Un professore ha costruito un programma che riceve in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisce in uscita la versione ordinata alfabeticamente dello stesso array.

b) E' decidibile stabilire se il generico array passato al programma sia già ordinato?

Es.20b

Un professore ha costruito un programma che riceve in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisce in uscita la versione ordinata alfabeticamente dello stesso array.

b) E' decidibile stabilire se il generico array passato al programma sia già ordinato?

Soluzione:

SI': basta verificare se l'array passato al programma coincide con la sua versione ordinata, ed esistono algoritmi noti che implementano l'ordinamento in modo corretto.

Es.20c

Un professore ha costruito un programma che riceve in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisce in uscita la versione ordinata alfabeticamente dello stesso array.

Per stabilire la correttezza del proprio programma (e di qualunque altro), il professore vuole sviluppare un programma «tester»: dato in ingresso il codice sorgente di un programma, il tester lo esegue con tutti i possibili (finiti!!) ingressi e controlla che il risultato sia sempre quello atteso.

c) Si può stabilire tramite il tester se un generico programma risponda alle specifiche?

Es.20c

Un programma che riceva in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisca in uscita la versione ordinata alfabeticamente dello stesso array.

c) Si può stabilire tramite un programma tester se un generico programma risponda alle specifiche?

Soluzione:

NO, per il teorema di Rice: la proprietà di ordinare correttamente l'array in ingresso è una proprietà non banale della funzione calcolata da un generico programma. E' non banale in quanto esiste una funzione calcolabile che ha la proprietà (es: quella di un qualunque algoritmo di ordinamento corretto) ed una funzione calcolabile che non ce l'ha (es: quella di un qualunque algoritmo che non calcoli l'ordinamento).

Es.20c

Un programma che riceva in ingresso un array di al più 5 stringhe di 3 caratteri costruite sull'alfabeto $\{a, b\}$ e che restituisca in uscita la versione ordinata alfabeticamente dello stesso array.

c) Si può stabilire tramite un programma tester se un generico programma risponda alle specifiche?

Osservazione:

Com'è possibile che non si possa costruire un tester nonostante gli array in ingresso da controllare siano in numero finito?

Perché ci potrebbe essere un qualche programma che su un certo ingresso entra in un loop infinito!

Es.21a

Data una generica NTM M e una generica stringa x, è **decidibile** stabilire se esistano almeno due cammini distinti di computazione effettuati da M che portano all'accettazione della stringa x?

Es.21a

Data una generica NTM M e una generica stringa x, è **decidibile** stabilire se esistano almeno due cammini distinti di computazione effettuati da M che portano all'accettazione della stringa x?

Soluzione:

NO, in quanto il problema dell'arresto, notoriamente non decidibile, è riducibile al problema in questione. Infatti:
Sia M' una generica NTM, e x una generica stringa. Costruiamo una nuova NTM M^x, uguale a M' tranne che per l'aggiunta di un cammino che a partire dallo stato iniziale accetta x in modo banale (ossia compiendo un passo verso un nuovo stato per ogni carattere di x).
Se potessimo decidere se M^x ammette almeno due cammini che accettano x, potremmo decidere se M' accetta x.

Es.21b

Data una generica NTM M e una generica stringa x, è **semidecidibile** stabilire se esistano almeno due cammini distinti di computazione effettuati da M che portano all'accettazione della stringa x?

Es.21b

Data una generica NTM M e una generica stringa x, è **semidecidibile** stabilire se esistano almeno due cammini distinti di computazione effettuati da M che portano all'accettazione della stringa x?

Soluzione:

SI', in quanto se esistono due cammini che accettano x, prima o poi li si troverà provando tutti i possibili cammini, attraversando in ampiezza (*breadth-first*) l'albero delle computazioni di M su input x.

Es.21c

Dato un generico NFSA A e una generica stringa x, è **decidibile** stabilire se esistano almeno due cammini distinti di computazione effettuati da A che portano all'accettazione della stringa x?

Es.21c

Dato un generico NFSA A e una generica stringa x, è **decidibile** stabilire se esistano almeno due cammini distinti di computazione effettuati da A che portano all'accettazione della stringa x?

Soluzione:

SI': basta enumerare tutti i (finiti) cammini di lunghezza $|x|$ su A a partire dallo stato iniziale e verificare se almeno due di essi accettano x.

Teoria Problemi di decisione noti

- Segue una lista di problemi di decisione che possono essere considerati noti in sede d'esame, ovvero la loro (non)(semi)decidibilità può essere data per nota senza bisogno di dimostrarla.
- Per ogni problema vengono fornite due versioni:
  - Una versione per TM come calcolatori di funzioni
  - Una versione per TM come riconoscitori di linguaggi
- Le due versioni (fornite sempre nell'ordine appena descritto) sono sempre equivalenti, nel senso che è possibile esibire una riduzione dell'una all'altra e viceversa.

Teoria Problemi di decisione noti

- **Problema dell'Arresto (Halting problem)**
  - «Dati una TM M_y ed un input x, è vero che $f_y(x) \neq \perp$?»
    - Input: y, x
    - Output: Sì se $f_y(x) \neq \perp$; No se $f_y(x) = \perp$
  - «Dati una TM M_y ed un input x, è vero che $x \in L(M_y)$?»
    - Input: y, x
    - Output: Sì se $x \in L(M_y)$; No se $x \notin L(M_y)$
  - Decidibilità:
    - Decidibile? NO → Particolariz. + Rice
    - Semidecidibile? SI' → Computazione di $M_y(x)$

Teoria Problemi di decisione noti

- **Problema della Emptiness**
  - «Data una TM M_y, è vero che $\forall x \in \mathbb{N}\left(f_y(x) = \perp\right)$?»
    - Input: y
    - Output: Sì se $\forall x\left(f_y(x) = \perp\right)$; No se $\exists x\left(f_y(x) \neq \perp\right)$
  - «Data una TM M_y, è vero che $L\left(M_y\right) = \emptyset$?»
    - Input: y
    - Output: Sì se $\forall x\left(x \notin L\left(M_y\right)\right)$; No se $\exists x\left(x \in L\left(M_y\right)\right)$
  - Decidibilità:
    - Decidibile? NO → Rice
    - Semidecidibile? NO → Il comlementare lo è (tramite enum. diag.)

Teoria Problemi di decisione noti

- **Problema della Totalità**
  - «Data una TM M_y, è vero che $\forall x \in \mathbb{N}\left(f_y(x) \neq \perp\right)$?»
    - Input: y
    - Output: Sì se $\forall x\left(f_y(x) \neq \perp\right)$; No se $\exists x\left(f_y(x) = \perp\right)$
  - «Data una TM M_y, è vero che $L\left(M_y\right) = \mathbb{N}$?»
    - Input: y
    - Output: Sì se $\forall x\left(x \in L\left(M_y\right)\right)$; No se $\exists x\left(x \notin L\left(M_y\right)\right)$
  - Decidibilità:
    - Decidibile? NO → Rice
    - Semidecidibile? NO → Diagonalizzazione

Teoria Problemi di decisione noti

- **Problema dell'Equivalenza**
  - «Date due TM $M_y, M_{y'}$, è vero che $f_y = f_{y'}$?»
    - Input: y, y'
    - Output: Sì se $\forall x \left(f_y(x) = f_{y'}(x)\right)$; No se $\exists x \left(f_y(x) \neq f_{y'}(x)\right)$
  - «Date due TM $M_y, M_{y'}$, è vero che $L(M_y) = L(M_{y'})$?»
    - Input: y, y'
    - Output: Sì se $L(M_y) = L(M_{y'})$; No se $L(M_y) \neq L(M_{y'})$
  - Decidibilità:
    - Decidibile? NO → Particolariz. + Rice
    - Semi-decidibile? NO → Riduzione da Emptiness

Teoria Problemi di decisione noti

Problema	Decidibile?	Semidecidibile?
Halting	No	Sì
Emptiness	No	No
Totalità	No	No
Equivalenza	No	No